

Invite Workflow: How It Works and What We Changed

Oct 6, 2026 · @QMAZ Holdings

Overview

An administrator invites a person by email from **Settings** → **Users**, and that person can only enter the system after choosing their own password and signing in with it. Between 2026-10-03 and 2026-10-06 we built and hardened this flow in six rounds, each started by something you noticed while testing.

Four parties take part:

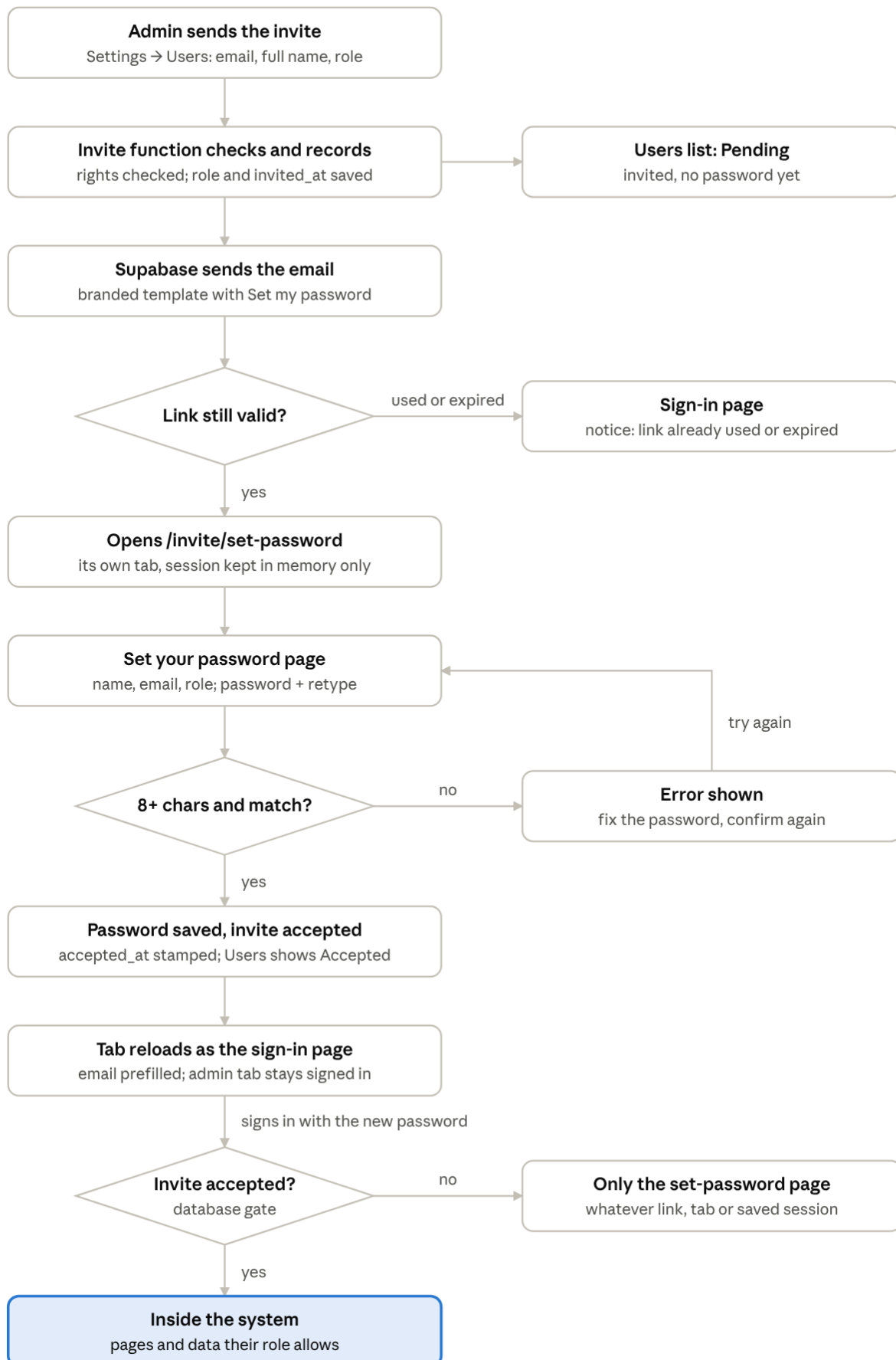
Party	What it does in the invite
Administrator	Fills in email, full name and role, then presses <i>Send invite</i>
Edge function <code>oe-invite-user</code>	Checks the admin may invite, asks Supabase Auth to send the email, saves the role on the profile
Supabase Auth	Sends the branded email, verifies the one-time link, sends the browser back to the app
The app on Vercel	Shows <i>Set your password</i> at <code>/invite/set-password</code> , saves the password, sends the person to sign in

The end result: the invited person sees who they are and what role they have, types a password twice, confirms, and lands on the sign-in page with their email filled in. The admin sees *Pending* until that happens and *Accepted* afterwards. Nobody reaches the system without a password, and an admin signed in on the same browser stays signed in.

Flow chart

Nobody enters the system before choosing a password

Invite flow, from Send invite to the first sign-in



invite flow · 8 steps, 3 decisions

The three decisions are where the fixes live: a used link stops at sign-in, a bad password loops back, and the database gate keeps anyone without a password out, however they arrive.

Timeline of changes

Each round started from something you saw while testing; the root cause was rarely where the symptom showed. The table is in the order it happened, so it reads as a story.

#	What you noticed	Root cause	Fix
1	Wanted a page showing the invited person and role, with password + retype, then a redirect to sign in	The page existed but showed no name or role, and after saving it went straight into the system	Set-password page shows name, email, role; <i>New password</i> + <i>Retype password</i> ; <i>Confirm password</i> signs out and opens sign-in with the email prefilled
2	The invite email was plain text; Users tab had no invite status	Supabase's default template; no columns recording invite or acceptance	Branded HTML email template; <code>invited_at</code> / <code>accepted_at</code> columns; <i>Pending</i> / <i>Accepted</i> column in Users
3	Invite link ended at "127.0.0.1 refused to connect"	<code>supabase config push</code> sent the whole auth section, overwriting the live Site URL with the local default	Repaired in the dashboard; <code>config.toml</code> now carries the real Vercel address
4	Clicking <i>Set my password</i> a second time opened the system with full access	First click left a saved session; second click had no invite marker, so the app treated the person as signed in	Server-side gate: invited but not accepted = only the set-password page; used-link message
5	Wanted a proper address instead of <code>/#...</code>	Single-page app had one path; the invite returned to <code>/</code>	New route <code>/invite/set-password</code> : Vercel rewrite, invite redirect, address bar kept in step

#	What you noticed	Root cause	Fix
6	Admin signed in on the same browser was logged out when the invitee confirmed	One saved session per browser; the invite replaced it, and sign-out was broadcast to all tabs	Invite tab keeps its session in memory only, then reloads as the sign-in page

Round 1: the set-password page

When the email link opens, Supabase puts `type=invite` and the session tokens after `#` in the address. The app reads that once at start-up and shows the page.

```
const INITIAL_HASH = window.location.hash;
const NEEDS_PASSWORD = /type=(invite|recovery)/.test(INITIAL_HASH);
```

The page loads the profile the admin created, so it can show *name, email and role* before the person types anything. Confirming calls `updateUser({ password })`, removes the tokens from the address bar, signs the invite session out and shows sign-in with "Your password is saved".

Round 2: branded email and invite status

The email is a Go template that Supabase fills in. It can only print what the invite sends, so the edge function now passes the role's label in the user's metadata.

```
{{ if .Data.full_name }}Hi {{ .Data.full_name }},{{ end }}
{{ .Email }} · {{ .Data.role_label }}
<a href="{{ .ConfirmationURL }}">Set my password</a>
```

Status needs two facts the database did not keep. The migration

`20261003000000_oe_invite_status.sql` adds them and a function the invited person calls on themselves:

Column	Set by	Meaning
<code>invited_at</code>	Sign-up trigger and the invite function	The invite was sent
<code>accepted_at</code>	<code>oe_accept_invite()</code> , right after the password is saved	The person chose a password

The Users table shows *Pending* when only `invited_at` is set, *Accepted* when `accepted_at` is set, and a dash for accounts the developer created directly.

Round 3: "127.0.0.1 refused to connect"

The address after `redirect_to=` decides where Supabase sends the browser. It had become the local-development default because `supabase config push` replaces every auth setting in `supabase/config.toml`, not only the email template. The fix was to restore Site URL and Redirect URLs in the dashboard and to write the real address into `config.toml`, so a future push cannot undo it.

```
site_url = "https://project-monitoring-expense.vercel.app"
additional_redirect_urls = ["https://project-monitoring-expense.vercel.app/**", "http://localhost:5173/**"]
```

Round 4: the second-click bypass

This was a security hole. The first click turned the one-time token into a session saved in the browser. The second click hit a used token, so Supabase returned `#error=...&error_code=otp_expired` instead of `type=invite`. The app found the saved session, saw no invite marker, and let the person in without a password.

The fix does not trust the address bar. It asks the database whether the invite was accepted, every time the app enters:

```
if (prof && prof.invited_at && !prof.accepted_at) {
  setAuth({ status: "set_password" });
  return;
}
```

A used link with no session now shows "That link was already used or has expired. Ask your administrator to send a new invite."

Round 5: the `/invite/set-password` route

A **route** is a path in the address that maps to one screen. This app has no router library, so two pieces make the route work:

1. `vercel.json` rewrites `/invite/set-password` to `index.html`, so Vercel serves the app instead of a 404.
2. The app asks Supabase to return invites to that path, and keeps the address bar in step: the path while setting a password, `/` everywhere else.

The Redirect URLs list in Supabase needs `/**` at the end, or Supabase rejects the path and falls back to the Site URL.

Round 6: the admin was logged out

Supabase saves the signed-in session in the browser's local storage, one per site, and tells every open tab when it changes. Opening the invite in the admin's browser replaced the admin's session; signing the invitee out then signed the admin out everywhere.

The invite tab is now isolated. Its Supabase client keeps the session in memory only, and when it is done it reloads itself as the normal sign-in page. The success message travels across that reload in `sessionStorage`, which belongs to that one tab.

```
const INVITE_TAB = INITIAL_PATH === INVITE_PATH || NEEDS_PASSWORD;
createClient(url, key, { auth: { persistSession: !INVITE_TAB, ... } });
```

How each round was checked

Every change was verified headlessly: the app was built against a small mock Supabase server and opened in a simulated browser (jsdom). The last run covered five cases, all passing: fresh invite with an admin signed in, the reloaded sign-in page, a used link, the bare address, and a pending person with a saved session. A real email end to end is still yours to try after deploying.

Components and files

The workflow lives in seven places; the app file holds the most logic, the database holds the facts the logic trusts.

File	Runs where	Its job in the invite
<code>supabase/functions/oe-invite-user/index.ts</code>	Supabase Edge Functions	Checks the caller has <code>settings.users</code> ; blocks admin-role invites by non-admins and duplicate emails; calls <code>inviteUserByEmail</code> with <code>full_name</code> , <code>role_label</code> and <code>redirectTo</code> ; saves role, <code>invited_at</code> on the profile
<code>supabase/migrations/20261003000000_oe_invite_status.sql</code>	Supabase database	Adds <code>invited_at</code> and <code>accepted_at</code> ; sign-up trigger copies the invite time; <code>oe_accept_invite()</code> ; backfills older invites
<code>supabase/templates/invite.html</code>	Supabase Auth email	The branded email: greeting, email and role card, <i>Set my password</i> button, fallback link, expiry note

File	Runs where	Its job in the invite
<code>supabase/config.toml</code>	Supabase CLI	Wires the email template; holds Site URL and Redirect URLs, which <code>config push</code> sends to the live project
<code>vercel.json</code>	Vercel	Rewrites <code>/invite/set-password</code> to the app
<code>src/App.jsx</code>	The browser	Everything the person sees (pieces below)
<code>docs/SETUP.md</code> , <code>README.md</code>	Documentation	Setup, deploy steps, the config-push warning

The pieces inside `src/App.jsx` :

Piece	What it does
<code>NEEDS_PASSWORD</code> , <code>LINK_ERROR</code>	Read once from the address at start-up: an invite link, or a used/expired link
<code>INVITE_PATH</code> , <code>INVITE_TAB</code>	The route <code>/invite/set-password</code> , and whether this tab exists only to set a password
<code>loadSupabase()</code>	Creates the Supabase client; the invite tab gets <code>persistSession: false</code>
<code>api.inviteUser</code>	Calls the edge function with <code>redirect_to</code> set to the invite route
<code>api.updatePassword</code>	Saves the password, then calls <code>oe_accept_invite</code>
<code>InviteForm</code>	The admin's <i>Invite user</i> dialog
Users table <i>Invite</i> column	<i>Pending</i> , <i>Accepted</i> or a dash, with dates on hover
<code>SetPassword</code>	The page: who you are, role chip, password + retype, Show/Hide, <i>Confirm password</i>
<code>enter()</code>	The gate: invited and not accepted = set-password page only
<code>passwordSet</code> / <code>handoffToSignIn</code>	Ends the invite session and reloads to sign-in, carrying the notice in <code>sessionStorage</code>
<code>Login</code> (<code>notice</code> prop)	Shows the one-time message and prefills the email

Deploy checklist

Three things are still open as of this writing: the migration is not pushed, the round 6 fix is not committed, and no real email has been tested end to end. The gate in round 4 and the *Pending / Accepted* column both read the migration's columns, so push it first.

- ☐ Push the database migration (adds `invited_at`, `accepted_at`, `oe_accept_invite`)
- ☐ Deploy the invite function (sends `role_label`, stamps `invited_at`)
- ☐ Commit and push `src/App.jsx` and `docs/SETUP.md` (the isolated invite tab) so Vercel redeploys
- ☐ Supabase → Authentication → URL Configuration: Site URL = `https://project-monitoring-expense.vercel.app`
- ☐ Same page, Redirect URLs: add `https://project-monitoring-expense.vercel.app/**`
- ☐ Supabase → Authentication → Emails → Templates → *Invite user*: paste `supabase/templates/invite.html`, subject "You're invited to Project Expense Monitoring" (or use `config push`, now safe because `config.toml` has the real address)
- ☐ Test: invite a spare email address, open the email in the admin's browser, set a password, confirm the admin stays signed in and the row turns *Accepted*
- ☐ Test: click the email button a second time and confirm it shows "already used or has expired"

```
supabase login
supabase link --project-ref <your project ref>
supabase db push
npm run functions:deploy
git add src/App.jsx docs/SETUP.md
git commit -m "Isolate the invite tab so the admin stays signed in"
git push
```

Later: custom SMTP in Supabase changes the sender from "Supabase Auth" to your own name and lifts the built-in limit of a few emails per hour.

Key concepts and lessons

Six ideas explain every bug we hit; knowing them lets you predict the next one.

Concept	What it means	Where it bit us
Session	Proof that a browser is signed in, kept as tokens. Supabase saves it in local storage, one per site, shared by every tab	Round 4: a leftover session let the person in. Round 6: the invite replaced the admin's session
One-time token	The code in the email link works once; after that Supabase returns an error instead of a session	Round 4: the second click looked different from the first, so the app's address-bar check missed it
Redirect URL	Where Supabase sends the browser after verifying a link. It must be on the allow list, or Supabase uses the Site URL instead	Round 3: pointed at <code>127.0.0.1</code> . Round 5: needed <code>/**</code> for the new path
Route and rewrite	A route maps an address to a screen. A single-page app has one real file, so the host must <i>rewrite</i> other paths to it	Round 5: without the Vercel rewrite, <code>/invite/set-password</code> would be a 404
Configuration as code	<code>config.toml</code> describes the live project; pushing it replaces everything it lists, not only what you changed	Round 3: pushing the email template also pushed a local Site URL
Trust the server, not the address bar	Anything in the URL can be missing or replayed. Decisions about access belong to data the server keeps	Round 4: the fix reads <code>accepted_at</code> from the database instead of looking for <code>type=invite</code>

Lessons worth keeping:

- **Test the second time, not only the first.** Every flow with a one-time link needs a test for the link used twice, used late, and opened with someone else already signed in.
- **Defence in depth.** The invite is now guarded twice: the isolated invite tab, and the database gate. Either alone would stop round 4.
- **Know the blast radius of a command.** Before running a deploy or push command, ask what else it sends.
- **One shared resource, two users, one browser** is a classic bug source. Here it was local storage; elsewhere it can be cookies or a cache.

- **Small, verified steps.** Each round changed one thing and was checked by an automated test before moving on.

Self-check questions

Try answering each one before reading the answer under it.

1. **Which part sends the invite email: the app, the edge function, or Supabase Auth?**
Supabase Auth sends it. The edge function only asks it to, through `inviteUserByEmail`, after checking permissions.
2. **Why can the email show the role, when Supabase Auth knows nothing about roles?**
The edge function puts the role's label into the user's metadata as `role_label`, and the template prints `{{ .Data.role_label }}`.
3. **What do *Pending* and *Accepted* each depend on?** *Pending* = `invited_at` set and `accepted_at` empty. *Accepted* = `accepted_at` set, written by `oe_accept_invite()` after the password is saved.
4. **The link says `redirect_to=http://127.0.0.1:3000`. Where do you look first?**
Supabase → Authentication → URL Configuration. Then ask what last changed it; here it was `supabase config push`.
5. **Why did checking for `type=invite` in the address not stop the second-click bypass?**
A used token returns an error in the address, not `type=invite`, while the saved session from the first click is still there.
6. **Why is the gate in `enter()` safer than any address-bar check?** It reads `accepted_at` from the database, which the visitor cannot fake or replay, and runs on every entry path.
7. **What two things make `/invite/set-password` work?** The Vercel rewrite to `index.html`, and `/**` in Supabase's Redirect URLs so the path is allowed.
8. **Why was the admin logged out, and why doesn't it happen now?** Both shared one saved session in local storage, and sign-out was broadcast to every tab. The invite tab now keeps its session in memory only.
9. **After confirming, why does the invite tab reload instead of just showing the sign-in form?** Its session is memory-only, so a sign-in there would vanish on refresh. Reloading turns it into a normal tab first.
10. **How does the "Your password is saved" message survive that reload?** It is stored in `sessionStorage`, which belongs to that tab, and read once by the sign-in page.
11. **What breaks if the migration is not pushed but the app is deployed?** The profile has no `invited_at` or `accepted_at`, so the gate never triggers and the Users column shows only dashes. If the new invite function is deployed too, saving the profile fails with "Invited, but the role could not be saved".